

# Solidify: Real-Time Static Analysis for Solidity in IDE

Nguyen Huu Canh<sup>1,2</sup>, Sit Khai Dong<sup>1,2</sup>, Tuan-Dung Tran<sup>1,2</sup>

<sup>1</sup>*University of Information Technology, Ho Chi Minh City, Vietnam*

<sup>2</sup>*Vietnam National University, Ho Chi Minh City, Vietnam*

*23520166@gm.uit.edu.vn, 23520299@gm.uit.edu.vn, dungtrt@uit.edu.vn*

## ABSTRACT

Smart contracts underpin decentralized finance (DeFi) and Web3 applications, stewarding billions of dollars in value. Security defects in Solidity code can cause catastrophic losses. While standard development environments provide robust syntax highlighting and compilation checks, they often lack immediate feedback on security vulnerabilities. This leaves a blind spot during the coding phase, forcing developers to rely on slow, external audits later in the pipeline. This paper introduces Solidify, a Visual Studio Code extension that delivers real-time security static analysis directly inside the IDE. The tool adopts a hybrid strategy: it leverages tree-sitter to construct rich Abstract Syntax Trees (ASTs) while complementing them with high-performance regular expressions for rapid pattern checks. A modular rule architecture makes it straightforward to extend coverage across security, syntax, and naming conventions. Our evaluation on 300 real-world smart contracts compares Solidify against the standard Solidity extension by Juan Blanco. Results show that Solidify complements the standard extension by catching security vulnerabilities that syntax-focused tools miss, achieving strong recall (0.91) and high precision (0.95), with an F1 score of 0.93 compared to 0.83 for compilation-based diagnostics. Solidify delivers instant feedback at 538 ms ( $\pm 19$  ms) -  $4.5\times$  faster than compilation-based diagnostics (2447 ms  $\pm 139$  ms), enabling developers to catch security issues as they type rather than waiting for compilation cycles. This demonstrates that integrating security awareness directly into the editor is both performant and essential for modern smart contract development.

**Keywords:** Smart Contracts, Solidity, Static Analysis, Blockchain Security, Visual Studio Code, Abstract Syntax Tree

## 1 Introduction

The rise of blockchain technology ushered in a new era of decentralized applications with smart contracts at the core. Solidity has become the de facto language for authoring these contracts on Ethereum and other EVM-compatible chains. Yet the immutability of blockchain - the very property that ensures trust, cuts both ways: once deployed, smart contract code cannot be patched. Seemingly minor vulnerabilities can be exploited to drain digital assets worth millions of dollars [1].

The stakes have never been higher. High-profile incidents such as the DAO hack (2016, \$50 million loss), the Parity wallet freeze (2017, \$280 million locked), and countless DeFi protocol exploits demonstrate that even well-funded projects with experienced teams fall victim to subtle security flaws. Each exploit erodes trust in the ecosystem and highlights a fundamental tension: developers must write flawless code on their first attempt, yet traditional development workflows offer minimal security guidance during the coding phase itself.

Standard IDE tools, such as the widely used Solidity extension by Juan Blanco [6], have greatly improved the developer experience by providing syntax highlighting, snippet generation, and compilation integration. However, these tools generally focus on correctness rather than security. They typically rely on the solc compiler for diagnostics, which means feedback is constrained by compilation times and limited to compiler errors. A developer can write an entire reentrancy vulnerability, see no warnings in their editor, successfully compile the contract, and only discover the flaw days later during a manual audit or-worse-after deployment when attackers exploit it in production.

This blind spot is not merely inconvenient; it is dangerous. Security analysis has traditionally been relegated to post-development phases: external audits, CI/CD pipeline checks, or manual review sessions. While tools like Slither [2], Mythril [4], and Securify [3] provide powerful vulnerability detection, they operate outside the immediate development loop. Developers must context-switch away from their code, run separate analysis tools, interpret results, and then return to their editor to apply fixes. This workflow friction means security checks happen infrequently - often only before major deployments rather than continuously during development.

We identify a critical gap: the lack of tools that raise security warnings with the same speed and convenience as syntax errors. Modern developers expect their IDEs to underline type mismatches in red as they type; why should critical security patterns like unchecked external calls or timestamp dependence be any different? To close this gap we built Solidify, a VS Code extension designed to be lightweight, responsive, and tightly integrated with the editor. Unlike standard extensions that wait for compilation, Solidify proactively scans for vulnerability patterns as the developer types, delivering security intelligence at the exact moment it is most valuable: during active code composition.

Our main contributions are:

- Hybrid analysis architecture: We present a design that combines AST-based reasoning with regex-based pattern detection to bypass the overhead of full compilation while maintaining detection accuracy.
- Modular rule system: We architect the analyzer so that rules can be developed, maintained, and extended independently, enabling rapid adaptation to emerging vulnerability patterns.
- Comparative evaluation: We demonstrate that Solidify enables instant security feedback as developers type, responding in 538 ms compared to 2447 ms for compilation-based tools - a 4.5 $\times$  improvement that eliminates the wait-compile-check cycle. This real-time responsiveness complements traditional syntax checking while achieving superior security detection ( $F1 = 0.93$  vs.  $0.83$ ).

## 2 Related Work

Security analysis of smart contracts has attracted significant research. Existing tools generally fall into three categories: Static Analysis, Dynamic/Symbolic Execution, and IDE Tooling.

**Static Analysis Frameworks:** These tools inspect source code without execution. Slither [2] converts Solidity to an intermediate representation (SlithIR) to run data-flow and taint analysis. It is fast for a CI tool but too slow for keystroke-level feedback. Securify [3], developed at ETH Zurich, employs formal verification using dependency patterns to prove compliance with security properties. While highly precise, Securify acts as a validator rather than a real-time assistant, requiring complete and compilable code to function effectively. SmartCheck [10] translates Solidity into an XML-based intermediate representation and applies XPath patterns to detect vulnerabilities, though it

similarly requires complete compilation before analysis.

**Symbolic and Dynamic Analysis:** Mythril [4] and Manticore [5] represent the heavy-weight class of analysis. Mythril uses concolic analysis to explore attackable states. Manticore, developed by Trail of Bits, is a symbolic execution framework that can navigate complex execution paths to find deep logic bugs. ZEUS [9] combines abstract interpretation with symbolic model checking to verify correctness and fairness properties of smart contracts. These tools are computationally expensive, often taking minutes or hours to run, making them suitable for final audits but impractical for the interactive development loop. Oyente [8], one of the pioneering tools in this space, performs symbolic execution to detect common vulnerabilities such as transaction-ordering dependence and timestamp dependence, but similarly suffers from high computational overhead.

**IDE Extensions:** The developer’s primary interface is the IDE. The most popular tool is Juan Blanco’s Solidity extension [6]. It wraps the official solc compiler to provide diagnostics. While it effectively catches syntax errors (e.g., missing semicolons, type mismatches), it does not natively scan for security patterns like reentrancy or timestamp dependence.

Solidify aims to bridge this gap by bringing lightweight security intelligence (inspired by static analyzers) into the immediate environment of the IDE (like Blanco’s extension).

Table 1: Comparison of Analysis Approaches

| <b>Tool</b>        | <b>Methodology</b>          | <b>Speed</b>   | <b>Target Phase</b> |
|--------------------|-----------------------------|----------------|---------------------|
| Oyente [8]         | Symbolic Execution          | Very Slow      | Deep Audit          |
| Slither [2]        | Static Analysis (IR)        | Moderate       | CI / Pre-commit     |
| Securify [3]       | Formal Verification         | Slow           | Audit               |
| SmartCheck [10]    | Pattern Matching (XML)      | Moderate       | Pre-deployment      |
| ZEUS [9]           | Abstract Interpretation     | Slow           | Audit               |
| Mythril [4]        | Concolic Analysis           | Very Slow      | Deep Audit          |
| Manticore [5]      | Symbolic Execution          | Very Slow      | Deep Audit          |
| J. Blanco Ext. [6] | Compiler Wrapper            | Fast           | Coding              |
| <b>Solidify</b>    | <b>Hybrid (AST + Regex)</b> | <b>Instant</b> | <b>Coding</b>       |

### 3 Methodology

Meeting real-time performance targets while maintaining accuracy requires a hybrid and modular design. Solidify blends two complementary analysis techniques that work in concert to provide immediate security feedback without sacrificing detection quality.

The first technique is AST-based analysis, which relies on the tree-sitter library [7] and the tree-sitter-solidity grammar. Unlike the standard extension which may wait for a full AST from the compiler, tree-sitter builds partial ASTs incrementally, allowing for sub-millisecond updates even on broken code. This incremental parsing capability is crucial for real-time feedback: developers can receive security warnings while typing incomplete statements, rather than waiting until the entire file compiles successfully.

The second technique is regex-based analysis, which complements the AST approach by providing instant detection of localized patterns. Regular expressions are used to immediately flag dangerous keywords such as `tx.origin` and `selfdestruct` without the need for semantic understanding. While less sophisticated than AST traversal, regex matching operates at nearly zero computational cost and can catch common anti-patterns before the AST is even constructed.

Solidify’s implementation follows a five-layer architecture designed for isolation and speed. Layer 1 - VS Code Integration anchors the tool to the IDE lifecycle, capturing document change events and managing the extension’s activation state. Layer 2 - Core Analysis Engine orchestrates the analysis pipeline, coordinating between parsing, rule execution, and diagnostic generation while managing performance through intelligent caching and debouncing strategies.

Layer 3 - AST/CFG Processing constructs structural views of the code using tree-sitter, building both abstract syntax trees and control flow graphs that serve as the foundation for deeper analysis. Layer 4 - Enhanced Rules Engine contains the security and quality rules themselves, organized into modular rule sets that can be independently developed, tested, and deployed. Each rule operates on the AST or applies regex patterns, emitting findings when vulnerability patterns are detected.

Finally, Layer 5 - Advanced Diagnostic Display surfaces findings to the user through VS Code’s native diagnostic system, providing inline warnings, quick-fix suggestions, and contextual documentation. This layered architecture ensures that performance bottlenecks can be isolated and optimized without affecting the entire system, while also enabling incremental feature development as new vulnerability patterns emerge.

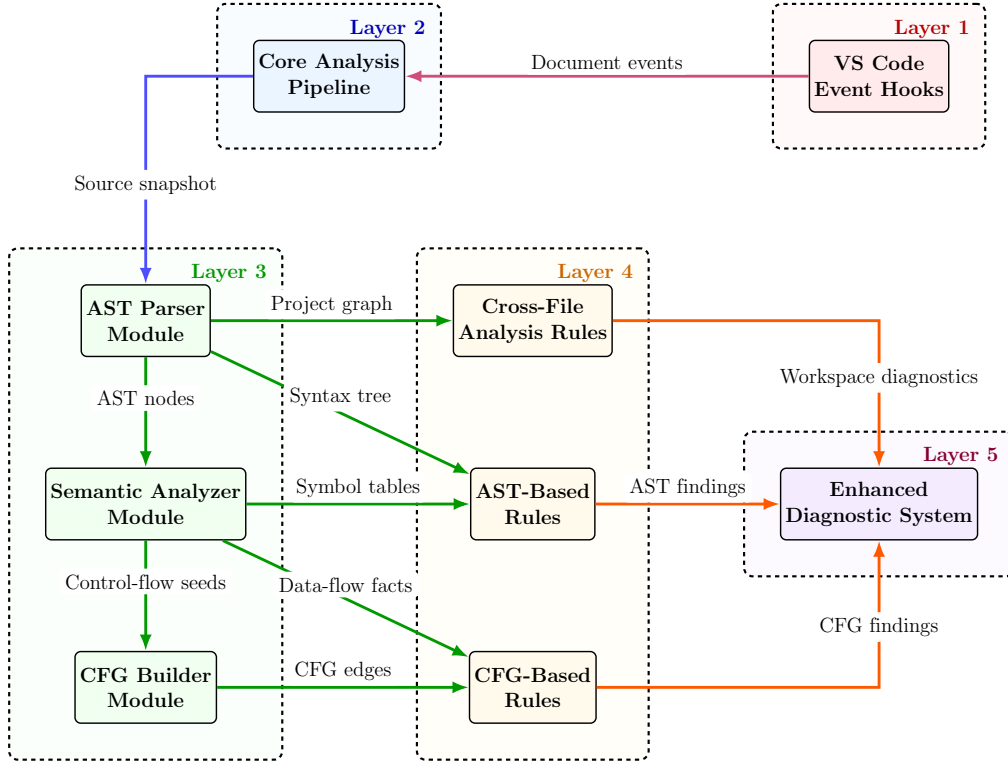


Figure 1: A five-layer design orchestrating document events from VS Code (Layer 1) through core analysis pipeline (Layer 2), AST/CFG processing (Layer 3), modular rule execution (Layer 4), to final diagnostic display (Layer 5). Data flows horizontally then vertically then horizontally again, with each layer performing specialized transformations before passing structured results downstream.

## 4 Experimental Results

We evaluate Solidify along two key dimensions: analysis latency and security detection accuracy. Our assessment compares Solidify against Juan Blanco’s Solidity extension (v0.0.176), the most widely adopted VS Code tool for Solidity development, which relies

on the solc compiler for diagnostics.

## 4.1 Experimental Setup

**Dataset:** We collected 300 real-world Solidity smart contracts of varying size and complexity, sampled from popular open-source projects on GitHub. The contracts range from simple token implementations to complex DeFi protocols, ensuring diversity in code patterns and potential vulnerability types.

**Baseline Configuration:** The baseline tool uses default solc compiler integration for diagnostics, representing the standard development experience for most Solidity developers.

**Hardware:** All experiments were conducted on identical hardware configurations (CPU: Intel i9-14900HX, RAM: 32 GB) to ensure fair comparison and reproducible results.

Table 2: Comparison of Solidify vs. Blanco Extension Performance and Accuracy

| Metric              | Solidify             | Blanco Extension       |
|---------------------|----------------------|------------------------|
| Analysis Latency    | 538ms ( $\pm 19$ ms) | 2447ms ( $\pm 139$ ms) |
| Precision           | 0.95                 | 1.00                   |
| Recall              | 0.91                 | 0.71                   |
| F1 Score            | 0.93                 | 0.83                   |
| False Positive Rate | 0.05                 | 0.00                   |

## 4.2 Response Latency

We measured the elapsed time from a keystroke event to the appearance of diagnostic feedback in the IDE interface. This metric directly impacts developer experience and workflow fluidity. In real-world usage, this latency difference determines whether security feedback feels instantaneous or disruptive.

Solidify achieves an average latency of 538 ms ( $\pm 19$  ms), whereas compilation-based diagnostics average 2447 ms ( $\pm 139$  ms). This  $4.5\times$  improvement fundamentally changes the development experience: developers see security warnings appear immediately as they type, without waiting for compilation to complete. The Blanco extension’s reliance on triggering the Solidity compiler introduces significant overhead through process spawning, full parsing, and type checking - each keystroke must wait for the entire compilation pipeline. In contrast, Solidify’s incremental tree-sitter parsing analyzes code changes in isolation, delivering feedback that feels instantaneous. This sub-second response time ensures the UI remains fluid and never interrupts the coding flow, transforming security analysis from a delayed batch process into a continuous, real-time companion.

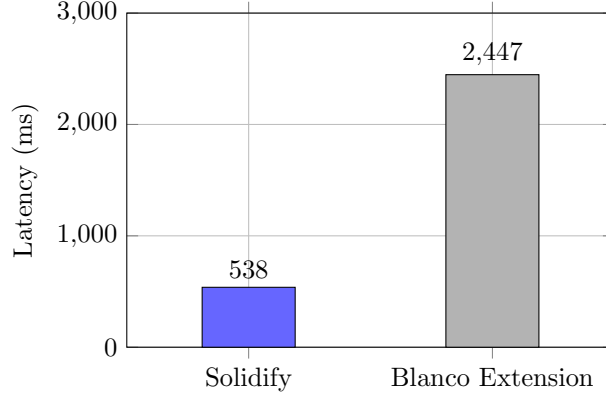


Figure 2: Average Diagnostic Latency Comparison

### 4.3 Accuracy Metrics

We evaluated both tools on their ability to identify known security vulnerabilities in our dataset, including reentrancy patterns, unchecked external calls, timestamp dependence, and unsafe authentication mechanisms.

The Blanco tool, designed primarily for syntax correctness, demonstrates strong precision (1.00) but limited recall (0.71), resulting in an F1 score of 0.83. While it successfully flags compilation warnings with zero false positives, it misses many security-specific patterns. The perfect precision reflects that flagged issues are indeed problems, but these are predominantly syntax errors rather than security vulnerabilities.

Solidify complements the baseline by focusing on security detection, achieving an F1 score of 0.93 with high precision (0.95) and high recall (0.91). The tool successfully catches nearly all security vulnerabilities in the dataset while maintaining a very low false positive rate (0.05). While not matching the exhaustive coverage of offline audit tools like Slither, ZEUS, or Manticore, Solidify represents a significant advancement over standard IDE capabilities, catching critical vulnerabilities during active development.

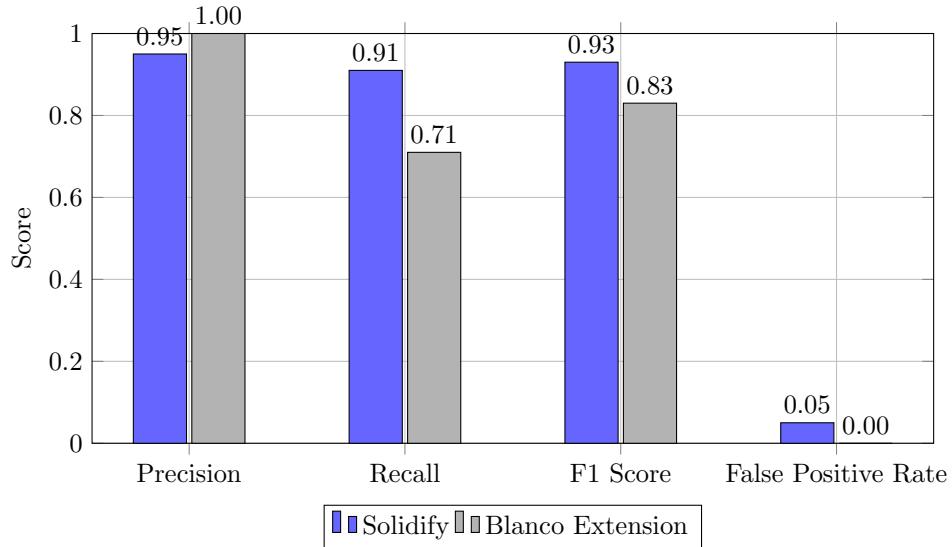


Figure 3: Comparative Analysis of Accuracy Metrics

## 5 Discussion

Our evaluation demonstrates that Solidify successfully addresses a critical gap in smart contract development tooling by bringing security-aware static analysis into the real-time IDE workflow without compromising performance. The most significant achievement is response latency: at 538 ms, Solidify delivers feedback  $4.5\times$  faster than compilation-based approaches, transforming security analysis from a disruptive wait into an instantaneous response. Developers no longer need to pause their workflow, trigger compilation, and wait for results - security warnings appear as they type, just like syntax errors. The hybrid AST-regex architecture proves particularly effective: tree-sitter’s incremental parsing delivers sub-second response times crucial for maintaining developer flow, while pattern-based rules catch common vulnerabilities instantly. This represents a paradigm shift from the traditional compile-check-fix cycle to continuous security feedback that never interrupts the creative process.

The tool’s modular rule architecture offers significant practical advantages. Security patterns can be independently developed, tested, and deployed without modifying the core analysis engine. This extensibility ensures that Solidify can evolve alongside emerging vulnerability classes and Solidity language updates. Furthermore, by integrating directly into the developer’s primary workspace, Solidify reduces context switching and lowers the barrier to security awareness - developers receive guidance at the moment they write potentially vulnerable code rather than discovering issues hours or days later during external audits.

Despite these strengths, Solidify has several limitations that constrain its current capabilities. The most significant is limited semantic depth: our hybrid approach excels at pattern matching but cannot perform deep data-flow analysis or symbolic reasoning. Complex vulnerabilities requiring cross-function taint tracking or multi-transaction attack scenarios remain beyond the tool’s detection scope. This means Solidify should complement, not replace, thorough security audits using comprehensive analysis frameworks.

The current rule coverage, while addressing common vulnerability patterns, is not exhaustive. Our evaluation focused on well-documented vulnerability classes, but the rapidly evolving DeFi ecosystem continuously produces novel attack vectors. The tool’s effectiveness depends on rule maintenance and expansion, requiring ongoing community contribution or dedicated curation. Additionally, the false positive rate of 0.05, while reasonable for a real-time tool, may cause alert fatigue if developers encounter frequent incorrect warnings during active development. Balancing sensitivity with precision remains an ongoing challenge, particularly for rules that require semantic context our lightweight analysis cannot fully capture.

## 6 Conclusion

This paper presented Solidify, a real-time static analysis extension that integrates security intelligence directly into the Solidity development workflow within Visual Studio Code. Our primary contributions include a hybrid AST-regex analysis architecture that bypasses compilation overhead, a modular rule system enabling independent extension of security checks, and empirical validation demonstrating practical viability. Experimental results show that Solidify delivers instant security feedback at 538 ms -  $4.5\times$  faster than the 2447 ms required for compilation-based diagnostics. This speed transforms the developer experience: security warnings appear as code is typed, eliminating the disruptive wait-compile-check cycle. Beyond speed, Solidify provides specialized security detection ( $F1 = 0.93$ ) that complements traditional syntax checking ( $F1 = 0.83$ ), catching vulnerabilities that compilation alone misses. These findings validate that shifting security analysis left to the earliest moment in the development lifecycle is

both technically feasible and practically valuable for smart contract development.

Looking forward, several promising directions exist for enhancing Solidify’s capabilities. To address deeper semantic analysis, we plan to integrate lightweight data-flow analysis that can track variable states across function boundaries without full symbolic execution overhead. Expanding rule coverage will involve establishing a community-driven rule repository with standardized testing frameworks to ensure quality and reduce false positives. We also intend to incorporate machine learning-based pattern recognition to detect novel vulnerability classes that lack explicit rules. Finally, cross-platform support beyond VS Code (such as JetBrains IDEs and Vim) and integration with CI/CD pipelines will extend Solidify’s reach across the entire development lifecycle. These enhancements will position Solidify as a comprehensive security companion for smart contract developers.

## References

- [1] N. Atzei, M. Bartoletti, and T. Cimoli, “A survey of attacks on Ethereum smart contracts,” in *Principles of Security and Trust (POST)*, ser. Lecture Notes in Computer Science, vol. 10260. Cham: Springer, 2017, pp. 109-139.
- [2] J. Feist, G. Grieco, and A. Groce, “Slither: A static analysis framework for smart contracts,” in *IEEE International Conference on Blockchain (Blockchain)*, 2019, pp. 8-15.
- [3] P. Tsankov, A. Dan, D. Drachsler-Cohen, A. Gervais, F. Bünzli, and M. Vechev, “Securify: Practical security analysis of smart contracts,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018, pp. 67-82.
- [4] B. Mueller, “Mythril: Security analysis tool for EVM bytecode,” *GitHub repository*, 2018. [Online]. Available: <https://github.com/ConsenSys/mythril>
- [5] M. Mossberg, F. Manzano, E. Hennenfent, A. Groce, G. Grieco, J. Feist, T. Brunson, and A. Dinaburg, “Manticore: A user-friendly symbolic execution framework for binaries and smart contracts,” in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019, pp. 1186-1189.
- [6] J. Blanco, “Solidity extension for Visual Studio Code,” *GitHub repository*, 2016. [Online]. Available: <https://github.com/juanfranblanco/vscode-solidity>
- [7] M. Brunsfeld, “Tree-sitter: An incremental parsing system for programming tools,” *GitHub repository*, 2018. [Online]. Available: <https://tree-sitter.github.io/tree-sitter/>
- [8] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, “Making smart contracts smarter,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 254-269.
- [9] S. Kalra, S. Goel, M. Dhawan, and S. Sharma, “ZEUS: Analyzing safety of smart contracts,” in *25th Annual Network and Distributed System Security Symposium (NDSS)*, 2018.
- [10] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, “SmartCheck: Static analysis of Ethereum smart contracts,” in *IEEE/ACM 1st International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2018, pp. 9-16.